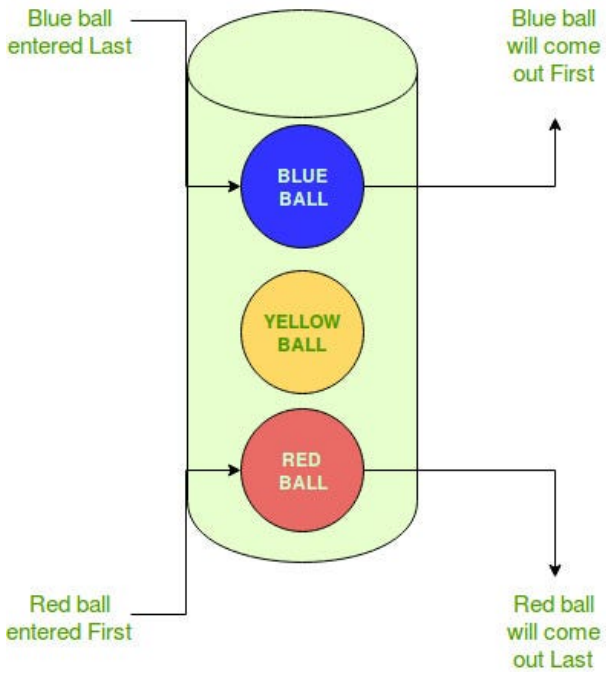
First In First Out - FIFO

Verification Plan



Prepared By: Eng. Jaber Darawsha

February 2026

Contents

1 Introduction **3**
1.1 What is First-In First-Out (FIFO)? . . . 3
1.2 Hardware Applications of FIFO . . . 3
1.3 Purpose of the Document . . . 3
1.4 Scope of Verification . . . 3
1.5 Assumptions / Constraints . . . 4
1.5.1 Constraints . . . 4
1.5.2 Assumptions . . . 4

2 Design Overview **5**
2.1 Key features and functionality . . . 5
2.2 Block diagram and interface . . . 6

3 Verification Strategy **8**
3.1 Methodology . . . 8
3.2 Selection of Verification Tests . . . 8

4 Testbench structure **10**

5 Tests **11**
5.1 Tests . . . 11
5.2 Coverage targets . . . 11
5.3 Testbench Architecture . . . 11
5.4 Tools and technologies . . . 12
5.5 Selection Of Verification Result . . . 12
5.6 Regression Tests . . . 14

List of Figures

2.1 Black box block diagram of the synchronous FIFO design. 6

3.1 An example of a FIFO with 5 items in the it. 9

4.1 UVM Testbench Structure. 10

5.1 UVM Classes Successfully Created. 12
5.2 FIFO Memory at Reset Signal Stage. 13
5.3 First cases after the reset signal. 13
5.4 Ignore Write Operation During Full FIFO. 13
5.5 Verifiy that the FIFO Read Data Successfully. 13
5.6 Verifiy that the FIFO empty flag is activated after read all elements. . . . 13
5.7 Write and Read with FIFO Empty. 14
5.8 Running Regression Test Correctly. 14
5.9 fifo_subscriber Coverage Collector. 14

Chapter 1

Introduction

1.1 What is First-In First-Out (FIFO)?

FIFO (First-In-First-Out) is a data structure where the first element added is the first one to be removed. This approach ensures that elements are processed in the exact order they were added.

1.2 Hardware Applications of FIFO

Some applications that reflect the importance of FIFO in hardware:

- **Buffering in Data Communication**: FIFOs are used in network interfaces and UARTs to temporarily store the incoming data before it is processed, ensuring no data is lost when the processor is busy.
- **Pipeline Processing in CPUs**: FIFOs are used between CPUs pipeline stages to store instructions or data temporarily, ensuring the CPU pipeline running efficiently when some stages are slower or have a problems.

1.3 Purpose of the Document

This document outlines a verification plan for a First In First Out (FIFO) designed to be part of a larger processor architecture. It aims to ensure that the FIFO data is written and read in the correct order, that no data is lost, and that the FIFO operates reliably under different conditions.

1.4 Scope of Verification

The scope of verification for the FIFO module is limited to top-level verification, treating the FIFO as a black box. We will focus on validating its inputs and outputs, such as output data, full and empty signals, without considering internal details. This ensures that the FIFO correctly maintains data order, handles full and empty conditions, and interfaces reliably with other hardware modules.

1.5 Assumptions / Constraints

1.5.1 Constraints

- No Constraints.

1.5.2 Assumptions

- The design is stable but not synthesizable.
- The design and verification environment uses UVM methodology.
- Functional coverage goals are achievable and measurable
- All external dependencies, such as clocks and resets, are provided correctly.

Chapter 2

Design Overview

This section will cover the Desing Under Test (DUT), including its specifications, description, key features, and functionality, and will introduce the block diagram of the design and its interface.

2.1 Key features and functionality

- **Write Data**: Writes occur on posedge of clk when write_enable is high and full is low.
- **Read Data**:Reads occur on posedge of clk when read_enable is high and empty is low.
- Reset clears the FIFO to empty state.
- **Status Monitoring**: Automatic monitoring of the FIFO status for full or empty condition.
- **Operation selection**: The FIFO uses control signals, called the "write_enable" and "read_enable," to select which operation to perform. The control unit decodes these signals and configures the FIFO to execute the corresponding operation, ensuring the correct storage of data.

2.2 Block diagram and interface

This section presents the block diagram of the DUT and its interface. It highlights key components, their interactions, and the input/output signals, providing a clear view of the design's structure and how it communicates with external modules for proper verification.

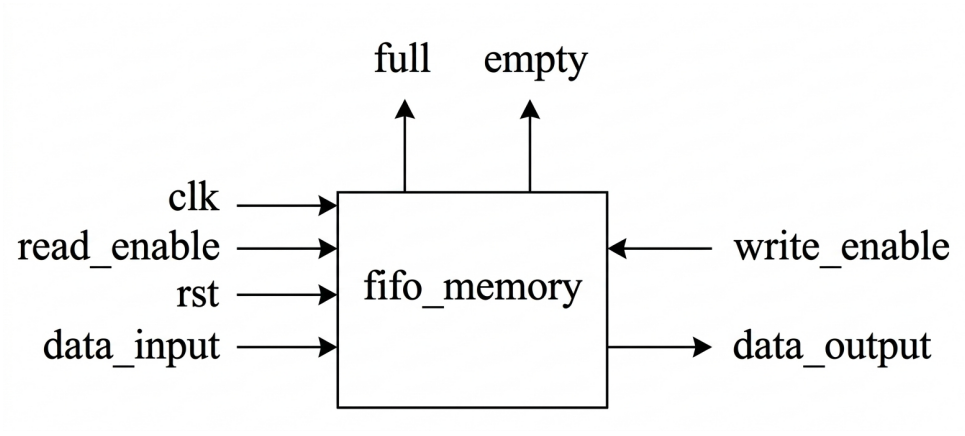


Figure 2.1: Black box block diagram of the synchronous FIFO design.

The interface and signals details are listed below in the table

Table 2.1: interface

Signal	**Width**	**Direction**	**Description**
clk	1	input	Clock signal to the FIFO module.
rst	1	input	clears all stored data and sets all status signals to default values.
data_input	8	input	Data to be stored into the FIFO.
write_enable	1	input	a control signal that allows writing data (store) into the FIFO when active.
read_enable	1	input	a control signal that allows reading data (retrieving) from the FIFO when active.
data_output	8	output	Represents the retrieved data from the FIFO when the read_enable is active.
full	1	output	A signal that indicates the FIFO cannot accept new data because it is completely full.
empty	1	output	an output flag that indicates the FIFO has no data to read.

Chapter 3

Verification Strategy

3.1 Methodology

The FIFO module will be verified using the Universal Verification Methodology (UVM), adopting a top-level, black box approach. A UVM testbench will include a sequencer, driver, and monitor to apply input stimulus (data_input with write_enable and read_enable) and capture output responses (data_output, full and empty).

Sequences will be designed to cover normal operation, boundary conditions, and error scenarios, while the scoreboard compares actual outputs with expected results to ensure data integrity and consistency. This methodology provides a structured, reusable, and automated framework for functional verification of the FIFO module.

3.2 Selection of Verification Tests

The verification tests are based on the FIFO specifications provided in the FIFO Design Specification document. It aims to verify that the FIFO operates correctly under all defined conditions for **write**, **read** operations and flag behavior (**full** and **empty**), ensuring that the FIFO behaves as expected before integration with the rest of the system.

- **Basic Write and Read Operation**:
  Write one data item in data_input at posedge of clk with write_enable = 1. Read it on data_out at posedge of clk with read_enable = 1.
- **Write with FIFO Full**:
  Writing data item after full is high. The FIFO for the 9th data written should be ignored (no overwrite).
- **FIFO Empty After 8th Items Read**:
  Verify FIFO empties correctly after reading all 8 entries.
- **Mixing Read and Write Operations**:
  Random writing and read at posedge of clk.
- **Read With FIFO Empty**:
  Reading data from the FIFO after empty is high. The FIFO returns undefined or holds the last value.

- **FIFO Full After 8th Items written**:
  Verify FIFO full flag triggered to high correctly after inserting all 8 entries.
- **Boundary Conditions**:
  Check that full and empty flags are never high simultaneously.
- **Mixed Cases**:
  Combine one or more cases together and check the system behavior.

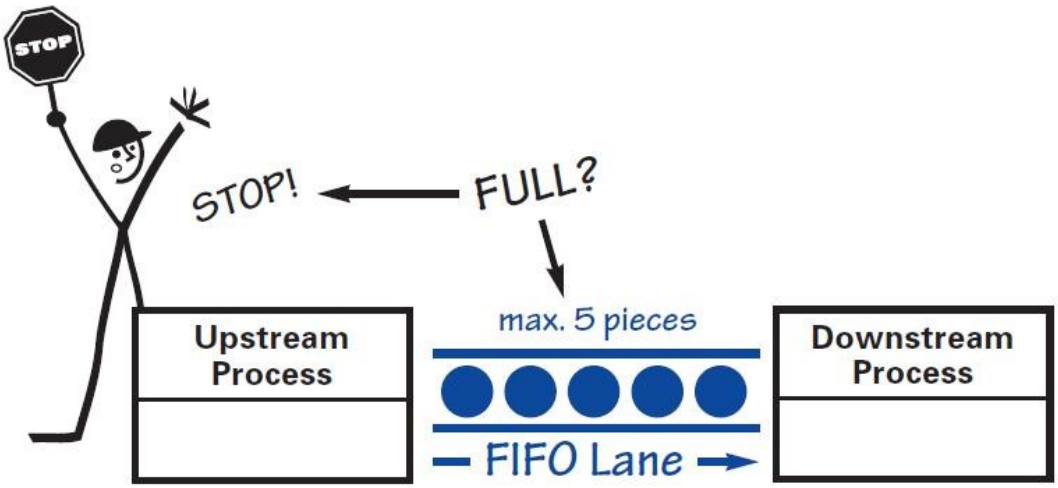


Figure 3.1: An example of a FIFO with 5 items in the it.

Chapter 4

Testbench structure

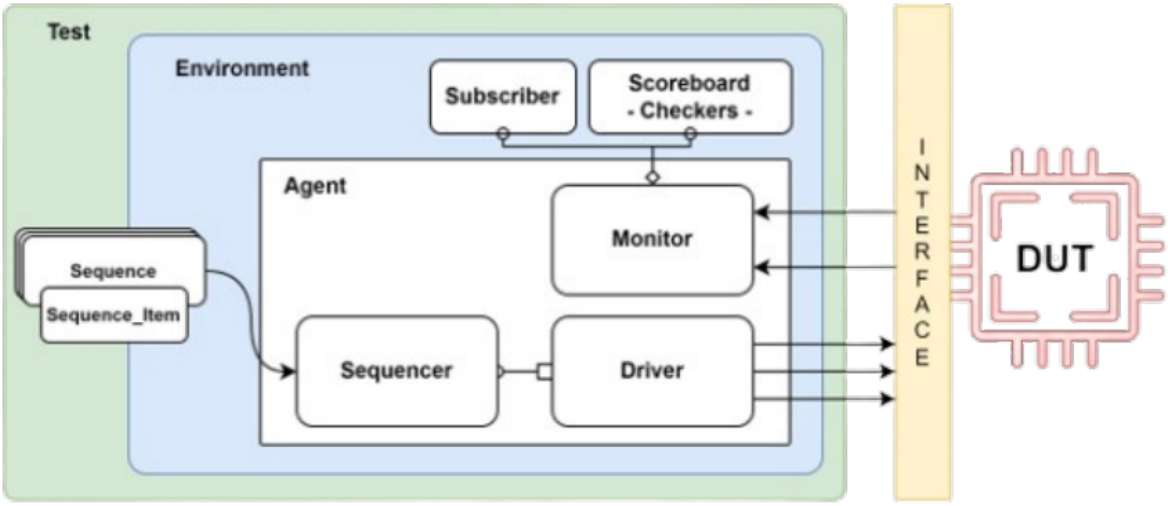


Figure 4.1: UVM Testbench Structure.

Chapter 5

Tests

5.1 Tests

A FIFO verification was implemented using the UVM methodology to ensure the correctness of all core operations, including reset, write, and read. The verification focused on checking the expected behavior under different conditions, such as empty and full FIFO, and handling edge cases and potential errors. The results demonstrated that all operations worked reliably and the design behaved as intended in all tested scenarios.

- **Reset**: operation correctly initializes the FIFO.
- **Write**: operations were verified to store data without loss or duplication.
- **Read**: operations were verified to retrieve data accurately and in order.

This verification ensures that the FIFO has been fully validated in all its operations before being handed over to the physical design team and integrated into the larger system-on-chip (SoC) design.

5.2 Coverage targets

- Code coverage.
- Functional coverage.
- Toggle coverage.
- Cross coverage.
- Path coverage.

5.3 Testbench Architecture

- **Components**:
  - uvm_sequence_items
  - uvm_sequence

  - uvm_sequencer
  - uvm_driver
  - uvm_monitor
  - uvm_agent
  - uvm_scoreboard
  - uvm_subscriber
  - uvm_environment
  - uvm_test
  - Top

- **Interface**: Encapsulates related signals together into one block to simplify driving and monitoring. Used by uvm_driver and uvm_monitor to send or observe signals from FIFO DUT.

5.4 Tools and technologies

- Siemens Questa 2025.2
- UVM 1.2
- Visual Studio code (VS Code).
- EDA Playgroud simulator

5.5 Selection Of Verification Result

For full details of the test plan, please click on the file name: **Test_Plan.pdf**.

- Verification Environment Initialization: The following simulation snapshot demonstrates the successful construction of all UVM components and the correct initialization of the FIFO to its default state after reset.

```
#   ************       IMPORTANT RELEASE NOTES         ************
#
#   You are using a version of the UVM library that has been compiled
#   with `UVM_NO_DEPRECATED undefined.
#   See http://www.eda.org/svdb/view.php?id=3313 for more details.
#
#   You are using a version of the UVM library that has been compiled
#   with `UVM_OBJECT_DO_NOT_NEED_CONSTRUCTOR undefined.
#   See http://www.eda.org/svdb/view.php?id=3770 for more details.
#
#       (Specify +UVM_NO_RELNOTES to turn off this notice)
#
# UVM_INFO /usr/share/questa/questasim/verilog_src/questa_uvm_pkg-1.2/src/questa_uvm_pkg.sv(277) @ 0: reporter [Questa UVM] QUESTA_UVM-1.2.3
# UVM_INFO /usr/share/questa/questasim/verilog_src/questa_uvm_pkg-1.2/src/questa_uvm_pkg.sv(278) @ 0: reporter [Questa UVM]  questa_uvm::init(+struct)
# UVM_INFO ./fifo_regression_test.sv(15) @ 0: uvm_test_top [fifo_regression_test] inside constructor of the fifo_regression_test class
# UVM_INFO @ 0: reporter [RNTST] Running test fifo_regression_test...
# UVM_INFO ./fifo_environment.sv(11) @ 0: uvm_test_top.env [fifo_environment] inside constructor of the fifo_environment
# UVM_INFO ./fifo_sequence_item.sv(14) @ 0: reporter@@item_ref [fifo_sequence_item] inside the constructor of the fifo_sequence_item
# UVM_INFO ./fifo_scoreboard.sv(19) @ 0: uvm_test_top.env.scoreboard [fifo_scoreboard] inside constructor of the fifo_scoreboard class
# UVM_INFO ./fifo_sequence_item.sv(14) @ 0: reporter@@fifo_sequence_item [fifo_sequence_item] inside the constructor of the fifo_sequence_item
# UVM_INFO ./fifo_subscriber.sv(21) @ 0: uvm_test_top.env.subscriber [fifo_subscriber] inside constructor of the fifo_subscriber
# UVM_INFO ./fifo_agent.sv(11) @ 0: uvm_test_top.env.agent [fifo_agent] inside constructor of the fifo_agent
# UVM_INFO ./fifo_monitor.sv(11) @ 0: uvm_test_top.env.agent.monitor [fifo_monitor] inside constructor of the fifo_monitor class
# UVM_INFO ./fifo_driver.sv(10) @ 0: uvm_test_top.env.agent.driver [fifo_driver] Inside the constructor of the fifo_driver class
# UVM_INFO ./fifo_reset_sequence.sv(7) @ 0: reporter@@reset_sequence [fifo_reset_sequence] Inside constructor of the reset_sequence class
# UVM_INFO ./fifo_sequence_item.sv(14) @ 0: reporter@@item [fifo_sequence_item] inside the constructor of the fifo_sequence_item
# UVM_INFO ./fifo_sequence_item.sv(14) @ 0: reporter@@item [fifo_sequence_item] inside the constructor of the fifo_sequence_item
# UVM_INFO ./fifo_sequence_item.sv(14) @ 7: reporter@@item [fifo_sequence_item] inside the constructor of the fifo_sequence_item
```

Figure 5.1: UVM Classes Successfully Created.

```
# ********************************************************************************************************************************************************** FIFO Memory **********************************************************************************************************************************************************
# wr_ptr = 0 | rd_ptr = 0 | elements_count = 0 | full = 0 | empty = 1
# [rd_ptr -> wr_ptr -> 0]    [0]    [0]    [0]    [0]    [0]    [0]    [0]
# ********************************************************************************************************************************************************************************************************************************************************************************************
```

Figure 5.2: FIFO Memory at Reset Signal Stage.

```
# UVM_INFO ./fifo_scoreboard.sv(49) @ 37: uvm_test_top.env.scoreboard [pass] -------- :: PASS:: --------
# UVM_INFO ./fifo_scoreboard.sv(54) @ 37: uvm_test_top.env.scoreboard [ACTUAL PACKET] Actual DUT Packet : rst = 1, data_input = 0, write_enable = 0, read_enable = 0, data_output = 0, full = 0, empty = 1
# UVM_INFO ./fifo_scoreboard.sv(55) @ 37: uvm_test_top.env.scoreboard [EXPECTED PACKET] EXpected DUT Packet : rst = 1, data_input = 0, write_enable = 0, read_enable = 0, data_output = 0, full = 0, empty = 1
# ************************************************************
# UVM_INFO ./fifo_write_sequence.sv(6) @ 40: reporter@@write_sequence [fifo_write_sequence] Inside constructor of the write_sequence class
# UVM_INFO ./fifo_sequence_item.sv(14) @ 40: reporter@@item [fifo_sequence_item] inside the constructor of the fifo_sequence_item
# UVM_INFO ./fifo_sequence_item.sv(14) @ 47: reporter@@item [fifo_sequence_item] inside the constructor of the fifo_sequence_item
#
# ********************************************************************************************************************************************************** FIFO Memory **********************************************************************************************************************************************************
# wr_ptr = 0 | rd_ptr = 0 | elements_count = 0 | full = 0 | empty = 1
# [rd_ptr -> wr_ptr -> 0]    [0]    [0]    [0]    [0]    [0]    [0]    [0]
# ********************************************************************************************************************************************************************************************************************************************************************************************
#
# UVM_INFO ./fifo_scoreboard.sv(49) @ 47: uvm_test_top.env.scoreboard [pass] -------- :: PASS:: --------
# UVM_INFO ./fifo_scoreboard.sv(54) @ 47: uvm_test_top.env.scoreboard [ACTUAL PACKET] Actual DUT Packet : rst = 0, data_input = 0, write_enable = 0, read_enable = 0, data_output = 0, full = 0, empty = 1
# UVM_INFO ./fifo_scoreboard.sv(55) @ 47: uvm_test_top.env.scoreboard [EXPECTED PACKET] EXpected DUT Packet : rst = 0, data_input = 0, write_enable = 0, read_enable = 0, data_output = 0, full = 0, empty = 1
# ************************************************************
# UVM_INFO ./fifo_sequence_item.sv(14) @ 50: reporter@@item [fifo_sequence_item] inside the constructor of the fifo_sequence_item
# UVM_INFO ./fifo_sequence_item.sv(14) @ 57: reporter@@item [fifo_sequence_item] inside the constructor of the fifo_sequence_item
#
# ********************************************************************************************************************************************************** FIFO Memory **********************************************************************************************************************************************************
# wr_ptr = 1 | rd_ptr = 0 | elements_count = 1 | full = 0 | empty = 0
# [rd_ptr -> 217]    [wr_ptr -> 0]    [0]    [0]    [0]    [0]    [0]    [0]
# ********************************************************************************************************************************************************************************************************************************************************************************************
#
# UVM_INFO ./fifo_scoreboard.sv(49) @ 57: uvm_test_top.env.scoreboard [pass] -------- :: PASS:: --------
# UVM_INFO ./fifo_scoreboard.sv(54) @ 57: uvm_test_top.env.scoreboard [ACTUAL PACKET] Actual DUT Packet : rst = 0, data_input = 217, write_enable = 1, read_enable = 0, data_output = 0, full = 0, empty = 0
# UVM_INFO ./fifo_scoreboard.sv(55) @ 57: uvm_test_top.env.scoreboard [EXPECTED PACKET] EXpected DUT Packet : rst = 0, data_input = 217, write_enable = 1, read_enable = 0, data_output = 0, full = 0, empty = 0
# ************************************************************
# UVM_INFO ./fifo_sequence_item.sv(14) @ 60: reporter@@item [fifo_sequence_item] inside the constructor of the fifo_sequence_item
# UVM_INFO ./fifo_sequence_item.sv(14) @ 67: reporter@@item [fifo_sequence_item] inside the constructor of the fifo_sequence_item
#
```

Figure 5.3: First cases after the reset signal.

```
# UVM_INFO ./fifo_scoreboard.sv(49) @ 137: uvm_test_top.env.scoreboard [pass] -------- :: PASS:: --------
# UVM_INFO ./fifo_scoreboard.sv(54) @ 137: uvm_test_top.env.scoreboard [ACTUAL PACKET] Actual DUT Packet : rst = 0, data_input = 56, write_enable = 1, read_enable = 0, data_output = 0, full = 1, empty = 0
# UVM_INFO ./fifo_scoreboard.sv(55) @ 137: uvm_test_top.env.scoreboard [EXPECTED PACKET] EXpected DUT Packet : rst = 0, data_input = 56, write_enable = 1, read_enable = 0, data_output = 0, full = 1, empty = 0
# ************************************************************
# UVM_INFO ./fifo_sequence_item.sv(14) @ 140: reporter@@item [fifo_sequence_item] inside the constructor of the fifo_sequence_item
# UVM_INFO ./fifo_sequence_item.sv(14) @ 147: reporter@@item [fifo_sequence_item] inside the constructor of the fifo_sequence_item
#
# ********************************************************************************************************************************************************** FIFO Memory **********************************************************************************************************************************************************
# wr_ptr = 0 | rd_ptr = 0 | elements_count = 8 | full = 1 | empty = 0
# [rd_ptr -> wr_ptr -> 217]    [115]    [101]    [6]    [28]    [160]    [12]    [181]
# ********************************************************************************************************************************************************************************************************************************************************************************************
#
# UVM_INFO ./fifo_scoreboard.sv(49) @ 147: uvm_test_top.env.scoreboard [pass] -------- :: PASS:: --------
# UVM_INFO ./fifo_scoreboard.sv(54) @ 147: uvm_test_top.env.scoreboard [ACTUAL PACKET] Actual DUT Packet : rst = 0, data_input = 98, write_enable = 1, read_enable = 0, data_output = 0, full = 1, empty = 0
# UVM_INFO ./fifo_scoreboard.sv(55) @ 147: uvm_test_top.env.scoreboard [EXPECTED PACKET] EXpected DUT Packet : rst = 0, data_input = 98, write_enable = 1, read_enable = 0, data_output = 0, full = 1, empty = 0
# ************************************************************
# UVM_INFO ./fifo_sequence_item.sv(14) @ 150: reporter@@item [fifo_sequence_item] inside the constructor of the fifo_sequence_item
# UVM_INFO ./fifo_sequence_item.sv(14) @ 157: reporter@@item [fifo_sequence_item] inside the constructor of the fifo_sequence_item
#
# ********************************************************************************************************************************************************** FIFO Memory **********************************************************************************************************************************************************
# wr_ptr = 0 | rd_ptr = 0 | elements_count = 8 | full = 1 | empty = 0
# [rd_ptr -> wr_ptr -> 217]    [115]    [101]    [6]    [28]    [160]    [12]    [181]
# ********************************************************************************************************************************************************************************************************************************************************************************************
```

Figure 5.4: Ignore Write Operation During Full FIFO.

```
# ********************************************************************************************************************************************************** FIFO Memory **********************************************************************************************************************************************************
# wr_ptr = 0 | rd_ptr = 1 | elements_count = 7 | full = 0 | empty = 0
# [wr_ptr -> 217]    [rd_ptr -> 115]    [101]    [6]    [28]    [160]    [12]    [181]
# ********************************************************************************************************************************************************************************************************************************************************************************************
#
# UVM_INFO ./fifo_scoreboard.sv(49) @ 207: uvm_test_top.env.scoreboard [pass] -------- :: PASS:: --------
# UVM_INFO ./fifo_scoreboard.sv(54) @ 207: uvm_test_top.env.scoreboard [ACTUAL PACKET] Actual DUT Packet : rst = 0, data_input = x, write_enable = 0, read_enable = 1, data_output = 217, full = 0, empty = 0
# UVM_INFO ./fifo_scoreboard.sv(55) @ 207: uvm_test_top.env.scoreboard [EXPECTED PACKET] EXpected DUT Packet : rst = 0, data_input = x, write_enable = 0, read_enable = 1, data_output = 217, full = 0, empty = 0
# ************************************************************
# UVM_INFO ./fifo_sequence_item.sv(14) @ 210: reporter@@item [fifo_sequence_item] inside the constructor of the fifo_sequence_item
# UVM_INFO ./fifo_sequence_item.sv(14) @ 217: reporter@@item [fifo_sequence_item] inside the constructor of the fifo_sequence_item
#
```

Figure 5.5: Verifiy that the FIFO Read Data Successfully.

- at FIFO full state, and then make just read operations for all elements, element by element, the empty should be activated and the memory should be cleared.

```
# ********************************************************************************************************************************************************** FIFO Memory **********************************************************************************************************************************************************
# wr_ptr = 0 | rd_ptr = 0 | elements_count = 0 | full = 0 | empty = 1
# [rd_ptr -> wr_ptr -> 217]    [115]    [101]    [6]    [28]    [160]    [12]    [181]
# ********************************************************************************************************************************************************************************************************************************************************************************************
#
# UVM_INFO ./fifo_scoreboard.sv(49) @ 277: uvm_test_top.env.scoreboard [pass] -------- :: PASS:: --------
# UVM_INFO ./fifo_scoreboard.sv(54) @ 277: uvm_test_top.env.scoreboard [ACTUAL PACKET] Actual DUT Packet : rst = 0, data_input = x, write_enable = 0, read_enable = 1, data_output = 181, full = 0, empty = 1
# UVM_INFO ./fifo_scoreboard.sv(55) @ 277: uvm_test_top.env.scoreboard [EXPECTED PACKET] EXpected DUT Packet : rst = 0, data_input = x, write_enable = 0, read_enable = 1, data_output = 181, full = 0, empty = 1
# ************************************************************
# UVM_INFO ./fifo_sequence_item.sv(14) @ 280: reporter@@item [fifo_sequence_item] inside the constructor of the fifo_sequence_item
# UVM_INFO ./fifo_sequence_item.sv(14) @ 287: reporter@@item [fifo_sequence_item] inside the constructor of the fifo_sequence_item
```

Figure 5.6: Verifiy that the FIFO empty flag is activated after read all elements.

In this case, when activating write_enable and read_enable at the same time with FIFO empty, the data_output should be undefined (read has higher priority than write operation).

```
#
# ********************************************************************************************************************************************************************** FIFO Memory **********************************************************************************************************************************************************
# wr_ptr = 1 | rd_ptr = 0 | elements_count = 1 | full = 0 | empty = 0
# [rd_ptr -> 107]    [wr_ptr -> 115]    [101]    [6]    [28]    [160]    [12]    [181]
# ********************************************************************************************************************************************************************************************************************************************************************************
#
# UVM_ERROR ./fifo_scoreboard.sv(51) @ 287: uvm_test_top.env.scoreboard [fail] --------:: FAIL :: --------
# UVM_INFO ./fifo_scoreboard.sv(54) @ 287: uvm_test_top.env.scoreboard [ACTUAL PACKET] Actual DUT Packet : rst = 0, data_input = 107, write_enable = 1, read_enable = 1, data_output = 181, full = 0, empty = 0
# UVM_INFO ./fifo_scoreboard.sv(55) @ 287: uvm_test_top.env.scoreboard [EXPECTED PACKET] EXpected DUT Packet : rst = 0, data_input = 107, write_enable = 1, read_enable = 1, data_output = 0, full = 0, empty = 0
# ************************************************************************
# UVM_INFO ./fifo_sequence_item.sv(14) @ 290: reporter@@item [fifo_sequence_item] inside the constructor of the fifo_sequence_item
# UVM_INFO ./fifo_sequence_item.sv(14) @ 297: reporter@@item [fifo_sequence_item] inside the constructor of the fifo_sequence_item
```

Figure 5.7: Write and Read with FIFO Empty.

5.6 Regression Tests

Repeatedly run previous test cases to ensure that new changes do not break existing functionality

```
#   ************       IMPORTANT RELEASE NOTES         *************
#
#   You are using a version of the UVM library that has been compiled
#   with `UVM_NO_DEPRECATED undefined.
#   See http://www.eda.org/svdb/view.php?id=3313 for more details.
#
#   You are using a version of the UVM library that has been compiled
#   with `UVM_OBJECT_DO_NOT_NEED_CONSTRUCTOR undefined.
#   See http://www.eda.org/svdb/view.php?id=3770 for more details.
#
#       (Specify +UVM_NO_RELNOTES to turn off this notice)
#
# UVM_INFO /usr/share/questa/questasim/verilog_src/questa_uvm_pkg-1.2/src/questa_uvm_pkg.sv(277) @ 0: reporter [Questa UVM] QUESTA_UVM-1.2.3
# UVM_INFO /usr/share/questa/questasim/verilog_src/questa_uvm_pkg-1.2/src/questa_uvm_pkg.sv(278) @ 0: reporter [Questa UVM]  questa_uvm::init(+struct)
# UVM_INFO ./fifo_regression_test.sv(15) @ 0: uvm_test_top [fifo_regression_test] inside constructor of the fifo_regression_test class
# UVM_INFO @ 0: reporter [RNTST] Running test fifo_regression_test...
# UVM_INFO ./fifo_environment.sv(11) @ 0: uvm_test_top.env [fifo_environment] inside constructor of the fifo_environment
# UVM_INFO ./fifo_sequence_item.sv(14) @ 0: reporter@@item_ref [fifo_sequence_item] inside the constructor of the fifo_sequence_item
# UVM_INFO ./fifo_scoreboard.sv(19) @ 0: uvm_test_top.env.scoreboard [fifo_scoreboard] inside constructor of the fifo_scoreboard class
# UVM_INFO ./fifo_sequence_item.sv(14) @ 0: reporter@@fifo_sequence_item [fifo_sequence_item] inside the constructor of the fifo_sequence_item
# UVM_INFO ./fifo_subscriber.sv(21) @ 0: uvm_test_top.env.subscriber [fifo_subscriber] inside constructor of the fifo_subscriber
# UVM_INFO ./fifo_agent.sv(11) @ 0: uvm_test_top.env.agent [fifo_agent] inside constructor of the fifo_agent
# UVM_INFO ./fifo_monitor.sv(11) @ 0: uvm_test_top.env.agent.monitor [fifo_monitor] inside constructor of the fifo_monitor class
# UVM_INFO ./fifo_driver.sv(10) @ 0: uvm_test_top.env.agent.driver [fifo_driver] Inside the constructor of the fifo_driver class
# UVM_INFO ./fifo_reset_sequence.sv(7) @ 0: reporter@@reset_sequence [fifo_reset_sequence] Inside constructor of the reset_sequence class
# UVM_INFO ./fifo_sequence_item.sv(14) @ 0: reporter@@item [fifo_sequence_item] inside the constructor of the fifo_sequence_item
# UVM_INFO ./fifo_sequence_item.sv(14) @ 0: reporter@@item [fifo_sequence_item] inside the constructor of the fifo_sequence_item
# UVM_INFO ./fifo_sequence_item.sv(14) @ 7: reporter@@item [fifo_sequence_item] inside the constructor of the fifo_sequence_item
```

Figure 5.8: Running Regression Test Correctly.

The following section presents the functional coverage results to verify that all required FIFO scenarios and signal combinations have been exercised during simulation.

```
# UVM_INFO ./fifo_regression_test.sv(54) @ 1040: uvm_test_top [fifo_regression_test] The end of the regression test
# UVM_INFO /usr/share/questa/questasim/verilog_src/uvm-1.2/src/base/uvm_objection.svh(1270) @ 1040: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
# UVM_INFO ./fifo_subscriber.sv(41) @ 1040: uvm_test_top.env.subscriber [fifo_subscriber] Coverage:  87.24%
# UVM_INFO /usr/share/questa/questasim/verilog_src/uvm-1.2/src/base/uvm_report_server.svh(847) @ 1040: reporter [UVM/REPORT/SERVER]
# --- UVM Report Summary ---
#
```

Figure 5.9: fifo_subscriber Coverage Collector.

End Verification Plan